COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/29/2026	**DATE SUBMITTED:**	7/29/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 **Array-Based vs. STL Stack Implementations**

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (std::stack).

Swap:

```
std::stack<char> stack1;

std::cout << "Is the stack empty? " << stack1.empty() << std::endl; // Check if the stack is empty
stack1.push('R'); // Push an element onto the stack
    std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stack
stack1.push('A'); // Push an element onto the stack
    std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stack
stack1.push('L'); // Push an element onto the stack
    std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stack
stack1.push('P'); // Push an element onto the stack
    std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stack
stack1.push('H'); // Push an element onto the stack
    std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stack

// C
std::cout << "Is the stack empty? " << stack1.empty() << std::endl; // Check if the stack is empty
std::cout << "Stack size: " << stack1.size() << std::endl; // Get the size of the stack
std::cout << "Top element: " << stack1.top() << std::endl; // Get the top element of the stackS

//swap
std::stack<char> stack2;
stack2.push('X');
stack2.push('Y');
stack2.push('Z');

// Swap the contents of the two stacks
stack1.swap(stack2);

std::cout << "After swapping:" << std::endl;
std::cout << "Stack 1 top element: " << stack1.top() << std::endl;
std::cout << "Stack 2 top element: " << stack2.top() << std::endl;
```

Observation:

After creating my first stack, stack1, I pushed the characters R, A, L, P, and H, which spell my name, RALPH. I then created another stack named stack2 and pushed the characters X, Y, and Z. Using the `swap()` function, I exchanged the entire contents of stack1 and stack2, not just their top elements.

After the swap, stack1 now contains the elements that were originally in stack2, so its top element becomes Z. Likewise, stack2 now contains the elements that were originally in stack1, making its top element H. This demonstrates that the `swap()` function exchanges all the elements between two stacks efficiently.

```cpp
//Emplace
stack1.emplace('A'); // Emplace an element onto the stack
std::cout << "After emplacing 'A' onto stack 1, top element: " << stack1.top() << std::endl;
```

emplace() works almost like push() due to the fact that the stack is holding only simple values, which are char type here. As you can see from the output of emplace('A'), a character 'A' was inserted right on the top of the stack1, where before that there was the character Z.

OUTPUTS:

```
Stack 1 top element: Z
Stack 2 top element: H
After emplacing 'A' onto stack 1, top element:
A
```

ILO B.1: Implement a stack using a Static Array.

Hoa4p1 > ILOB1 > main.cpp > push()

```cpp
#include <iostream>

// Global Variables
#define maxSize 10
// const int maxSize = 10;

int stackArr[maxSize];
int top = -1, newData;

// Function Prototypes
// Top, Push, Pop, isEmpty, isFull
bool isEmpty();
bool isFull();
void Top();             // The T is capital
void displayAll();      // Goal is to display the stack following the LIFO principle

// Transformer operators
void push();
void pop();

int main() {
    int choice;

    while (true) {
        std::cout << "\n================================================\n";
        std::cout << "Stack Operations\n";
        std::cout << "================================================\n";
        std::cout << "[1] Push\n";
        std::cout << "[2] Pop\n";
        std::cout << "[3] IsEmpty\n";
        std::cout << "[4] IsFull\n";
        std::cout << "[5] Top\n";
        std::cout << "[6] Display All\n";
        std::cout << "================================================\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {
```

```
        switch (choice) {

            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                std::cout << "Is the stack empty? " << isEmpty() << std::endl;
                break;

            case 4:
                std::cout << "Is the stack full? " << isFull() << std::endl;
                break;

            case 5:
                Top();
                break;

            case 6:
                displayAll();
                break;

            default:
                std::cout << "Invalid Choice!" << std::endl;
                break;
        }
    }

    return 0;
}

// Function Definitions

```

```
74
75  bool isEmpty() {
76      return (top == -1);
77  }
78
79  bool isFull() {
80      return (top == maxSize - 1);
81  }
82
83  void Top() {
84
85      // Error checking
86      if (isEmpty()) {
87          std::cout << "Stack is empty!" << std::endl;
88          return;
89      }
90
91      // Print the top element
92      std::cout << "The top element is: " << stackArr[top] << std::endl;
93  }
94
95  void push() {
96
97      // Error Checking
98      if (isFull()) {
99          std::cout << "Stack Overflow!" << std::endl;
100         return;
101     }
102
103     // Ask the user what they want to insert
104     std::cout << "Enter a number to push: ";
105     std::cin >> newData;
106
107     // Store the data in the stack
108     stackArr[++top] = newData;
109 }
110
111 void pop() {
```

```
112
113     // Error Checking
114     if (isEmpty()) {
115         std::cout << "Stack Underflow!" << std::endl;
116         return;
117     }
118
119     // Display the item to be popped
120     std::cout << "Popped element: " << stackArr[top] << std::endl;
121
122     // Remove the element from the stack
123     top--;
124 }
125
126 void displayAll() {
127
128     // Error Checking
129     if (isEmpty()) {
130         std::cout << "Stack is empty!" << std::endl;
131         return;
132     }
133
134     // Display the stack following the LIFO principle
135     std::cout << "\nStack Contents (Top to Bottom)\n";
136
137     for (int i = top; i >= 0; i--) {
138         std::cout << stackArr[i] << std::endl;
139     }
140 }
```

OUTPUT:

Push:

```
==========================================================
Stack Operations
==========================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==========================================================
Enter your choice: 1
Enter a number to push: 1
```

Pop and Stack Underflow:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 2
Popped element: 1

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 2
Stack Underflow!
```

IsEmpty

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 3
Is the stack empty? 1
```

Stack Overflow

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Stack Overflow!

==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 1
Stack Overflow!
```

IsFull

```
================================================
Stack Operations
================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
================================================
Enter your choice: 4
Is the stack full? 0

================================================
Stack Operations
================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
================================================
Enter your choice: 1
Enter a number to push: 1

================================================
Stack Operations
================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
================================================
Enter your choice: 4
Is the stack full? 0
```

Top:

```
================================================
Stack Operations
================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
================================================
Enter your choice: 5
The top element is: 1
```

Display All:

```
==================================================
Stack Operations
==================================================
[1] Push
[2] Pop
[3] IsEmpty
[4] IsFull
[5] Top
[6] Display All
==================================================
Enter your choice: 6

Stack Contents (Top to Bottom)
1
1
1
1
1
1
1
1
1
1
```

Analysis:

The program illustrates how stack can be implemented through a fixed-sized array with a maximum size of 10 elements. The program consists of various functions such as Push, Pop, IsEmpty, IsFull, Top and Display All. The variable used for storing the top element is "top". The value "-1" of "top" means that the stack is empty. Stack overflow and stack underflow error checks have been incorporated in the program.

The push() operation appends a new element at the top of the stack while the pop() operation deletes the latest inserted element into the stack by employing the LIFO (Last In First Out) rule. Top() operation displays the top element of the stack without deleting it. displayAll() operation displays all the elements of the stack from top to bottom.

Observation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization and code formatting**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Everything was done in the laboratory.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.